

16 — Caricare quando serve: IntersectionObserver, code e frame

Un interruttore che bloccava la pagina, e i tre motivi diversi per cui la bloccava. Come si fa a chiedere dati *quando l'utente li sta guardando*, come si inserisce un migliaio di nodi senza far scattare lo scorrimento, e perché `requestAnimationFrame` — la scelta istintiva — era la scelta sbagliata. Esempi: `src/ui/griglia-collezione/griglia-collezione.js`, `src/data/completamento.js`.

1. Il guasto

Nella collezione c'è un interruttore, «Mostra anche le carte che mi mancano»: accanto alle tue carte compaiono in grigio quelle che ti mancano per completare il set. Con il filtro «Set» impostato funzionava bene. Senza filtro, la pagina si piantava.

Il codice era questo — una riga, in fondo al disegno di ogni sezione-set:

```
if (this.#mostraMancanti && confrontabile(set)) this.#aggiungiMancanti(griglia, set);
```

Innocua a leggerla, perché non dice **quante volte** viene eseguita. La risposta è: una per ogni set di cui possiedi almeno una carta. Con una collezione sparsa su sessanta set, quella riga fa partire sessanta carichi nello stesso istante, e ognuno finisce per costruire duecento nodi DOM.

Vale la pena separare i tre costi, perché si curano in modi diversi:

costo	quanto	si cura con
sessanta fetch di file-set (30-400 KB l'uno)	decine di MB	non chiederli tutti
il JSON.parse di quei file	sincrono, sul thread principale	non chiederli tutti
~12.000 article costruiti con innerHTML	secondi di thread occupato	inserirli a pezzi

Il terzo è il vero colpevole. La rete è lenta ma **asincrona**: mentre scarica, la pagina risponde. Costruire dodicimila nodi è lento e **sincrono**: finché non finisce, il browser non dipinge e non raccoglie i tuoi tocchi. È la differenza fra un'attesa e un blocco.

Chi viene da Angular ha un riflesso pronto: «metti la paginazione, o una `*cdkVirtualFor`». Il ragionamento è esatto, e il resto di questo documento è come si fa a mano — perché qui non c'è un CDK da importare.

2. IntersectionObserver: chiedere al browser «è a schermo?»

La domanda «questo elemento è visibile?» si può rispondere da soli con `getBoundingClientRect()` dentro un gestore di `scroll`. È una pessima idea, per due motivi che vale la pena conoscere:

1. `getBoundingClientRect()` costringe il browser a ricalcolare il layout **in quel momento** (*forced synchronous layout*, o *layout thrashing*): il browser aveva in coda dei calcoli da fare pigramente, e tu pretendi la risposta subito;
2. l'evento `scroll` arriva a raffica, sul thread principale, esattamente mentre l'utente sta scorrendo — cioè nel momento in cui non devi rubare tempo.

`IntersectionObserver` sposta il lavoro **fuori** dal thread principale: è il motore di composizione a sapere già dove stanno le cose, e ti chiama solo quando uno degli elementi osservati entra o esce da un'area. La forma è sempre questa:

```
const osservatore = new IntersectionObserver(
  (voci) => {
    for (const voce of voci) {
      if (!voce.isIntersecting) continue; // ti chiama anche all'uscita
      /* ... */
    }
  },
  { rootMargin: '100px' }, // «entrando» comincia 100px prima
);
osservatore.observe(elemento);
```

Tre dettagli che si pagano se si ignorano:

- **la callback riceve un array**, non un elemento: se dieci sezioni entrano insieme, è **una** chiamata con dieci voci;
- **ti chiama anche quando le cose escono**, con `isIntersecting: false`. Da qui il `continue`;
- **rootMargin allarga l'area**. Nel progetto ce ne sono due valori diversi, e la differenza è una scelta di merito: 200px per le immagini delle card (precaricare un'immagine da 14 KB in anticipo è quasi gratis) e 100px per le sezioni-set (là dietro c'è un file di set intero: meglio arrivarci più vicini).

Nel progetto

La sezione non chiede più niente da sé: si fa mettere in coda.

```
if (this.#mostraMancanti && confrontabile(set)) {
  sezione._set = set; // il set viaggia sull'elemento
  this.#osserva(sezione);
}
```

Quel `sezione._set` merita una parola. Quando l'osservatore chiamerà, l'unica cosa che avrà in mano è `voce.target`, cioè il nodo DOM: il `set` di quel giro di `#disegnaSet()` sarà svanito da tempo. Appoggiare l'oggetto **sull'elemento** è il modo di far arrivare un dato a un callback che riceverà solo il nodo. È lo stesso trucco già usato per `card._voce`, e in un progetto senza framework è la sostituzione naturale di ciò che in Angular sarebbe un campo del componente di riga. Da Java sembra un abuso — stai aggiungendo un campo a un oggetto altrui — e in effetti lo è: gli oggetti JS sono aperti, e la disciplina la mette la convenzione (l'underscore) invece del compilatore.

3. Una fila, perché «visibile» a volte è tanta roba

L'osservatore non basta da solo, e il motivo è un caso concreto: se possiedi **una carta sola** di trenta set diversi, ogni sezione è alta poco più della sua card. A schermo ce ne stanno cinque o sei, e con il `rootMargin` di più. Le sveglierebbe tutte nello stesso istante, e saremmo di nuovo al punto di partenza, solo con numeri più piccoli.

La soluzione sta in una riga, e sfrutta una proprietà delle promesse che si dimentica facilmente: `.then()` **su una promessa già risolta non esegue subito, mette in coda**. Quindi una promessa tenuta come campo diventa una fila:

```
/** @type {Promise<void>} */
#coda = Promise.resolve();

// nella callback dell'osservatore:
this.#coda = this.#coda.then(() => this.#aggiungiMancanti(griglia, set, attesa));
```

Ogni nuovo lavoro si aggancia in fondo all'ultimo, e parte solo quando quello prima ha finito — perché `#aggiungiMancanti` è `async`, quindi la promessa che restituisce si risolve alla fine, non all'inizio. Cinque righe invece di un semaforo con contatore.

Attenzione a due cose:

- **se un lavoro in fila lancia, la fila muore.** #coda diventerebbe una promessa respinta e ogni `.then()` successivo la propagherebbe senza eseguire niente. Per questo `#aggiungiMancanti` ha il suo `try/catch` interno: un set illeggibile offline non deve zittire tutti i set dopo di lui;
- **il segnaposto si mette all'ingresso in fila, non all'inizio del lavoro.** Altrimenti una sezione in attesa del proprio turno sembrerebbe semplicemente completa — e «non ti manca niente» è una bugia diversa da «sto guardando».

4. Inserire a blocchi, e il tranello di `requestAnimationFrame`

Restava il terzo costo: un set può avere 250 carte, e costruirle in un colpo occupa il thread per centinaia di millisecondi — un *long task*, nel vocabolario degli strumenti di misura. Si spezza in blocchi, cedendo il controllo fra l'uno e l'altro:

```
for (let i = 0; i < mancanti.length; i += BLOCCO_MANCANTI) {
  if (i > 0) await cediIlPasso();
  if (!griglia.isConnected) return;      // filtro cambiato mentre inseriamo
  griglia.append(/* ... 60 card ... */);
}
```

La domanda interessante è **cosa** sia `cediIlPasso()`. La prima versione era questa, e sembra la risposta da manuale:

```
const prossimoFrame = () => new Promise((r) => requestAnimationFrame(r));
```

È sbagliata per due motivi, e uno l'ha trovato la prova sul campo.

Primo: `requestAnimationFrame` **non scatta in una scheda in secondo piano**. Il browser non disegna ciò che nessuno guarda, quindi non chiama i callback di animazione. Misurando il comportamento a scheda nascosta, l'inserimento si fermava esattamente al primo blocco — 60 card su 61 — e, essendo in fila, teneva bloccati anche tutti i set dopo. Su una PWA installata al telefono, dove si cambia app continuamente, è un guasto vero.

Secondo: `raf` **gira prima del disegno**, non dopo. Le card costruite là dentro pesano comunque su quel frame: si è spezzato il lavoro, ma non lo si è tolto dalla strada del disegno.

La versione buona cede al **ciclo degli eventi**, non al frame:

```
const cediIlPasso = () =>
  globalThis.scheduler?.yield?.() ?? new Promise((risolvi) => setTimeout(risolvi, 0));
```

`scheduler.yield()` è l'API nata per questo mestiere: interrompe, lascia lavorare il browser (disegno, tocchi, tastiera) e riprende **con priorità**, così il tuo lavoro non finisce in fondo a una coda dove chiunque può passarli davanti. Dove non c'è ancora, `setTimeout(..., 0)` fa la stessa cosa in modo grezzo: rimanda al prossimo giro del ciclo degli eventi. Nota l'`?.()` doppio — `scheduler?.yield?.()` — che copre sia il browser senza `scheduler` sia quello che lo ha con solo `postTask`.

Chi arriva da Java riconosce il paesaggio ma non le regole. `Thread.yield()` cede fra thread *paralleli*; qui il thread è uno solo, e cedere significa «spezza il mio compito in due compiti, e fra i due lascia passare il browser». Il ciclo degli eventi non ti *sospende*: ti *riplanifica*. Ed è per questo che ogni `await` è un punto in cui il mondo può essere cambiato — da cui i due `if (!griglia.isConnected)` `return`;

5. Come si è verificato

Un numero che non si misura è un'opinione. La prova è stata: seminare una collezione di **una carta in ognuno di 60 set diversi** (il caso peggiore descritto al §3), poi accendere l'interruttore contando le card apparse, le richieste di rete, e i *long task* con un `PerformanceObserver`:

```
const po = new PerformanceObserver((l) => {
  for (const e of l.getEntries()) if (e.duration > 50) lunghi.push(Math.round(e.duration));
});
po.observe({ entryTypes: ['longtask'] });
```

Con il tetto di 50 ms non tanto per convenzione quanto per il motivo dietro la convenzione: oltre quella soglia un tocco può aspettare abbastanza da farsi notare.

Esito: fermi in cima alla pagina, **2 set caricati su 60** (124 card), zero task oltre i 50 ms, il click gestito in 7 ms. Ed è così che si è scoperto il difetto di rAF: il primo tentativo, misurato a scheda nascosta, si fermava a 60 card con 56 segnaposti «cerco le carte che mancano...» piantati per sempre.

6. Cosa resta fuori

Due cose, dette perché non passino per finite:

- `#disegnaRisultati()` **rifà tutto l'elenco a ogni lettera** battuta nella casella di ricerca. Con mille carte in collezione diventerà lento da solo, interruttore o no. La cura è un'altra (rimandare il ridisegno di qualche decina di millisecondi, oppure riusare i nodi invece di ricrearli), e non c'entra con questo documento;
 - **la lista che si passa al visore** (`#apri()`) raccoglie tutte le card a schermo. Con le mancanti caricate su molti set diventa un array di migliaia di elementi: oggi innocuo, perché sono riferimenti e non copie.
-

7. Verifica

1. `IntersectionObserver` chiama la callback anche quando un elemento **esce** dall'area. Nel codice del progetto c'è un `continue` che serve solo a questo: cosa accadrebbe, in concreto, se lo togliessi? (Suggerimento: pensa a cosa fa `unobserve` un attimo dopo.)
2. La fila del §3 è una promessa riassegnata: `this.#coda = this.#coda.then(...)`. Se dentro `#aggiungiMancanti` togliessi il `try/catch` e un set fosse illeggibile, quanti set smetterebbero di caricarsi — quello, o tutti quelli dopo? Perché?
3. Perché `cediIlPasso()` non usa `requestAnimationFrame`? Dai i **due** motivi, e di' quale dei due si vede solo provando l'app.
4. Prova a cambiare `BLOCCO_MANCANTI` da 60 a 1 e poi a 500, misurando i long task come al §5. Cosa peggiora nei due casi opposti? (Uno dei due non peggiora affatto la fluidità: quale, e cosa peggiora invece?)